PODCAST CONTENT CONTROL PLANE

Full-Stack Metadata Management & Ingestion Engine

PROJECT VISION: A CONTENT CONTROL PLANE

A robust **content control plane** prototype designed to ingest, normalize, and govern third-party catalog data. It empowers teams with **data sovereignty** over external vendor feeds.

Durable Records

Maintain a canonical, queryable copy of metadata independent of vendor changes or throttling.

Stable Identifiers

Decouple product logic from volatile external IDs through a unified internal schema.

Full Accountability

Trace every modification through append-only audit logs and comprehensive sync histories.

WHY ORGANIZATIONS NEED A METADATA PLANE

DATA SOVEREIGNTY

Maintaining a canonical record even when vendors return bad payloads or throttle access. Ground truth is **ours**, not the vendor's.

POLICY HOOKS

Centralizing curation rules like **pinning, promoting, or hiding** content in a dedicated internal governance layer.

OPERATIONAL TRUST

Building confidence through **transparent sync run histories** and verifiable audit trails for compliance and support.

SYSTEM ARCHITECTURE: A VERTICAL SLICE

EXTERNAL API

iTunes Search API (or Mock) as the data source.

→

GO BACKEND

Gin-powered API with clean service/repo layers.

→

POSTGRES

Durable storage for normalized metadata.

→

REACT UI

Vite-based console for operator governance.

Conscious V1 Scope

Deliberate choice of **on-demand sync** and **in-memory TTL caching** to maximize performance while maintaining architectural simplicity.

Composition Root

`main.go` handles clean dependency injection, ensuring the system is **interview-friendly** and easily testable via mocks.

THE SYNCHRONIZATION ENGINE: INGEST & NORMALIZE

01 **Ingest Path**

Validates search terms and coordinates with the iTunes API (or mock) to retrieve raw metadata.

02 **Normalization**

Transforms diverse external payloads into a consistent, strictly typed internal domain model.

03 **Atomic Upsert**

Uses stable `source_id` keys to ensure data integrity and prevent duplicates during ingestion.

04 **Audit Integration**

Every sync event triggers an audit entry, creating a permanent record of system state changes.

Stable Identifiers

Decoupling internal product logic from volatile external IDs to ensure long-term system stability and reliable curation.

Cache Management

Automatically invalidates read caches upon successful sync to ensure the operator console remains accurate and fresh.

DATA PERSISTENCE: RELIABLE GROUND TRUTH

Postgres Authority

Internal Ownership

Tables for `podcasts`, `sync_runs`, and `audit_logs` serve as the definitive internal "ground truth," independent of vendor responses.

Performance Pooling

Utilizes `pgx/pgxpool` for efficient connection management and high-concurrency support during ingestion.

Atomic Upserts

Ensures data integrity by upserting valid rows based on stable `source_id` keys.

Rich Metadata Schema

Non-Trivial Normalization

Captures complex payloads including collection IDs, authors, genres, feed URLs, and high-res artwork.

Chronological Insights

Records are ordered by `updated_at`, providing operators with immediate visibility into data freshness.

Audit Integration

Every persistence event is linked to an append-only audit trail for full system accountability.

DEMO FLOW: FROM INGESTION TO AUDIT

*"I'm running **docker compose up --build**. Stack comes up: Postgres → Migrations → Go API (8080) → Vite UI (5173)."*

STEP	ACTION	KEY NARRATIVE
01	**Open UI at :5173**	"Operators use the same stable JSON API as automation; the UI is just a client."
02	**Run Sync (e.g. "news")**	"Ingest path: Search → iTunes (or mock) → Normalize → Upsert by stable source_id."
03	**Browse & Filter Catalog**	"Read path: We attach curation (pinned) without losing vendor refresh on metadata."
04	**Pin Show & View Audit**	"Curation writes audit events; append-only trail for support and compliance narratives."
05	**Health & API Check**	"Contract is stable for scripts and UI. Ground truth is ours, not the vendor response."

FRONTEND CONSOLE: OPERATOR INTERFACE

Unified Client Architecture

Built with **React and Vite**, the UI acts as a client to the same stable JSON API used by automation.

REACT 18 | VITE | TYPESCRIPT | AXIOS

Data Translation Layer

A dedicated `api.ts` layer handles seamless conversion between backend snake_case DTOs and frontend camelCase types.

Catalog Governance

Browse, filter, and "pin" critical shows. Real-time UI updates reflect curation changes immediately in the catalog view.

Sync Orchestration

Trigger manual synchronizations and monitor progress through integrated audit and sync history views.

Freshness Indicators

Visual classification of records as **fresh, aging, or stale** based on their last update timestamp.

OPERATIONAL VISIBILITY: MONITORING & TRUST

STABLE CONTRACTS

The **JSON API** serves as a consistent contract for both the operator UI and external automation scripts.

- **API Stability**
- **Automation Ready**

HEALTH MONITORING

Dedicated **/health** endpoint provides real-time liveness probes for infrastructure monitoring and automated recovery.

- **Liveness Probes**
- **Dependency Checks**

AUDIT NARRATIVES

Append-only trails support **compliance and support** narratives by documenting exactly who changed what and when.

- **Verifiable History**
- **Operator Attribution**

DOMAIN VERSATILITY: BEYOND PODCASTS

Music & Rights

Labels, ISRCs, and territory rules—ingest partner feeds and normalize to **your** playout schema with integrated rights management.

Video & FAST

Centralizing channel and series metadata from multiple aggregators into a single, unified plane for streaming platforms.

E-commerce

Marketplace listings governance; maintaining **your** canonical SKU and policy flags independent of vendor updates.

Ads & Sponsored Content

Third-party creative catalogs with integrated **approval and audit** workflows to ensure brand safety and compliance.

TESTING STRATEGY: ENSURING RELIABILITY

INTEGRATION FLOW TESTS

TestFlow_MockITunesWorksOffline

Verifies SearchPodcasts returns valid-looking shows without real HTTP dependencies.

TestFlow_SyncUpsertsMockShows

Validates the full service-to-repo sync flow, ensuring successful record upserts.

TestFlow_LivenessAlwaysOK

Confirms the /health endpoint remains responsive even with nil dependencies.

TestFlow_SyncWithoutQueryIs400

Ensures the API correctly handles malformed client input with proper 400 error codes.

INGEST MAPPING TESTS

TestMap_AppleJSON_ToCatalogRow

Verifies precise normalization of vendor fields to internal stable source_id keys.

TestMap_AppleJSON_ZeroCollectionDropped

Ensures invalid or empty vendor records are skipped to maintain catalog quality.

Sanitization Logic

Validates title trimming and "Unknown" artist mapping for incomplete vendor payloads.

TECHNICAL EXCELLENCE & MATURITY

01 Mature Delivery

A complete **vertical slice** from external API to UI, backed by targeted tests and documented tradeoffs, ensuring the project is both interview-friendly and production-ready.

02 Resilient Integration

The iTunes client handles **timeouts and retries**, with a robust mock mode for reliable demonstrations and offline testing without external dependencies.

03 Clean Codebase

Adheres to **modern best practices** in Go and React, with clear separation of concerns and a modular architecture that supports rapid onboarding and maintainability.

04 Scalable Design

Designed for growth, the architecture allows for easy expansion to **new data sources** and more complex governance policies through its flexible ingestion engine.

FUTURE ROADMAP: SCALING THE PLANE

OPERATIONAL HARDENING

Scheduled Orchestration

Transitioning to cron-based syncs and Cloud Scheduler for automated catalog updates.

Resilience Patterns

Implementing Circuit Breakers and Bulkheads to isolate external API failures from the read path.

Advanced Observability

Integrating OpenTelemetry for structured tracing and metrics for sync duration and success rates.

ARCHITECTURAL EVOLUTION

Distributed Caching

Leveraging Redis for cross-replica caching with advanced mitigation for cache stampedes.

Data Model Depth

Expanding to episode-level cataloging (v2) with full-text search via Postgres tsvector or Typesense.

Enterprise Readiness

Implementing RBAC/ABAC for granular operator roles and multi-tenant isolation.

STRATEGIC AI INTEGRATION: GOVERNANCE

METADATA ENRICHMENT

Using AI to suggest missing genres and descriptions, with **human-in-the-loop** approval before persistence to canonical fields.

- **Operator Approvals**
- **Data Completeness**

INTELLIGENT DEDUPLICATION

Assisting operators by identifying potential duplicate records with **confidence scores** for manual merging and catalog cleanup.

- **Catalog Integrity**
- **Confidence Scoring**

RESPONSIBLE GUARDRAILS

Ensuring AI never writes directly to production without validation, maintaining the integrity of the **internal ground truth**.

- **Validation Loops**
- **Risk Mitigation**

FOUNDATION FOR INCREMENTAL GROWTH

Proven Workflow

V1 establishes the core boundaries, audit trails, and operator interfaces needed for robust catalog governance.

Defensible Design

Future enhancements build upon the existing skeleton of clean layering, mockable boundaries, and internal data ownership.

Vision for Maturity

Moving from a vertical slice to a production-hardened platform while preserving data sovereignty and operational trust.